Name of the module: vector-scalar operation
Target delivery date:
Location in GIT repository: vspa_common\lib\vector

Overview:

This module computes y = x + alpha and y = x * alpha for different types of x, alpha, y, where x and y are vectors, alpha is a scalar.

Function API:

“vAddSclr” family:
static inline void vAddSclr(__fx16* py, __fx16* px, __fx16* palpha, size_t L)
static inline void vAddSclr(__fp16* py, __fp16* px, __fp16* palpha, size_t L)
static inline void vAddSclr(float* py, float* px, float* palpha, size_t L)

static inline void vAddSclr(vspa2_complex_fixed16* py, vspa2_complex_fixed16* px, vspa2_complex_fixed16* palpha, size_t L)
static inline void vAddSclr(vspa2_complex_float16* py, vspa2_complex_float16* px, vspa2_complex_float16* palpha, size_t L)
static inline void vAddSclr(vspa_complex_float32* py, vspa_complex_float32* px, vspa_complex_float32* palpha, size_t L)

“vMultiSclr” family:
static inline void vMultiSclr(__fx16* py, __fx16* px, float* palpha, size_t L)
static inline void vMultiSclr(__fp16* py, __fp16* px, float* palpha, size_t L)
static inline void vMultiSclr(float* py, float* px, float* palpha, size_t L)

static inline void vMultiSclr(vspa2_complex_fixed16* py, vspa2_complex_fixed16* px, float* palpha, size_t L)
static inline void vMultiSclr(vspa2_complex_float16* py, vspa2_complex_float16* px, float* palpha, size_t L)
static inline void vMultiSclr(vspa_complex_float32* py, vspa_complex_float32* px, float* palpha, size_t L)

API description:

" vAddSclr " contains a family of vector-scalar addition functions for different input/output data types. “vMultiSclr” contains a family of vector-scalar multiplication functions for different input/output data types.

Table 1: Function input and output of “vAddSclr” and “vMultiSclr”.

Input	
px	Pointer to x, a vector. Vector-aligned.
palpha	Pointer to alpha, a scalar. Vector-aligned.
L	The number of input DMEM lines. L >= 1.
Output	
py	Pointer to y, a vector. Vector-aligned.

DMEM requirements:

Table 2: DMEM requirements of “vAddSclr” and “vMultiSclr”.

Variable	Minimum size (words)	Minimum alignment (words)	Allocated by
y	32	32 (vector-aligned on 16AU)	Caller
x	32	32	Caller
alpha	1 halfword for half-precision; 1 word for single-precision.	half-word	Caller
L	1	1	Caller

Implementation:

“S2straight” allows 6 “vAddSclr” instances share one loop body for both real and complex data. “S1straight” allows 3 “vMultiSclr” instances share one loop body as long as alpha is real. The 9 instances can share one loop body.

Test plan:

Random data.
Test non-vector-aligned alpha.